Android 隐私一致性审计实验汇报

Lab2：Data Safety 与 Privacy Policy 自动比对

李禹江，张博涵，蒋兴宇

我的刀盾队

2026 年 3 月 31 日

目录

1. 实验背景
2. 代码实现与工程化
3. 实验结果
4. 合规解释与讨论
5. 结论与改进

实验背景

- **实验任务**
  - 对比 Android 应用 `Data Safety` 与 `Privacy Policy`
  - 利用 LLM 自动识别合规风险标签
- **输出标签（三分类）**
  - `incorrect`：Data Safety 未声明，但隐私政策提到
  - `incomplete`：Data Safety 有声明，但细节不完整
  - `inconsistent`：两者都提到，但描述冲突
- **实验价值**
  - 构建“模型初筛 + 人工复核”的隐私审计流程
  - 将输出结果映射到你们既有 22 项合规框架

环境与运行入口

- **运行环境**
  - OS：Windows（PowerShell）
  - Python：3.10（Conda 环境：`android_privacy_lab`）
- **核心依赖**
  - `transformers` / `torch` / `pandas` / `scikit-learn` / `matplotlib`
- **统一入口脚本**
  - `run_audit.py` 提供四个子命令：`audit` / `postprocess` / `evaluate` / `full`

Prompt 怎么写（代码中的真实模板）

```
SYSTEM_PROMPT = "You are an expert in analyzing..."

USER_PROMPT_TEMPLATE = """
Classify three labels:
1) incorrect ...
2) incomplete ...
3) inconsistent ...

Output format strictly as JSON:
{"incorrect": 0 or 1, "incomplete": 0 or 1, "inconsistent": 0 or 1}
Return JSON only.

Data Safety:
{data_safety}
Privacy Policy:
{privacy_policy}
"""
```

- **设计思路**：角色约束（system）+ 任务约束（user）+ 格式约束（JSON only）
- **工程设置**：temperature=0，降低随机性，提高可复现性

五个模块与代码函数映射

模块	核心函数/类	职责说明
Provider 层	`BaseProvider` / `make_provider`	统一 `infer()` 接口，屏蔽模型差异
Audit 层	`run_audit(...)`	逐行读取 CSV，调用 `provider.infer()`，写入 `result` 字段
Postprocess 层	`extract_json_dict` + `postprocess_results`	解析模型文本并标准化为三列 0/1 标签
Evaluate 层	`evaluate_results(...)`	计算 Precision/Recall/F1/Accuracy，输出指标 CSV 与混淆矩阵
可恢复运行	`count_data_rows` + `--resume`/`--resume-from`	自动/手动续跑，避免大任务中断后重复计算

输入输出如何处理（I/O 规范）

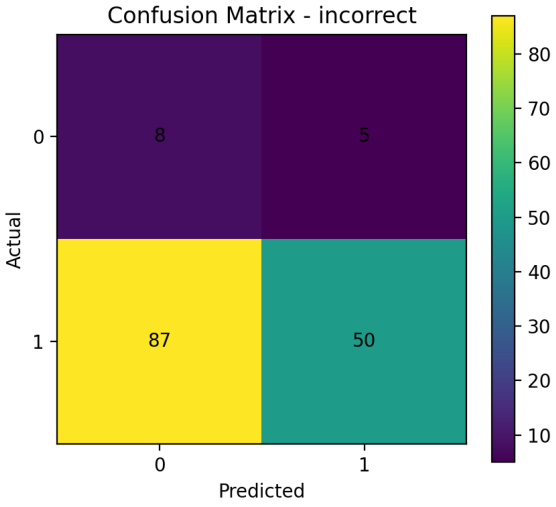
阶段	输入	输出
audit	`data_safety_content` + `privacy_policy_content`	`result={"incorrect":x,...}`
postprocess	含 `result` 的原始 CSV	三列标签：`incorrect/incomplete/inconsistent`
evaluate	预测 CSV + groundtruth CSV	`metrics_summary.csv` + confusion matrix 图

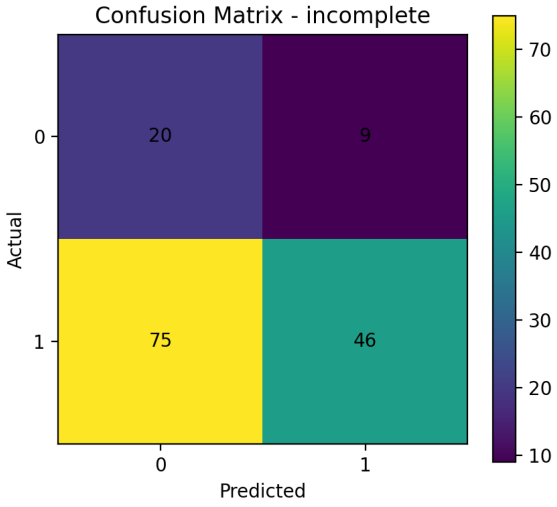
- **鲁棒性处理**：解析失败默认全 0，保证流程不中断

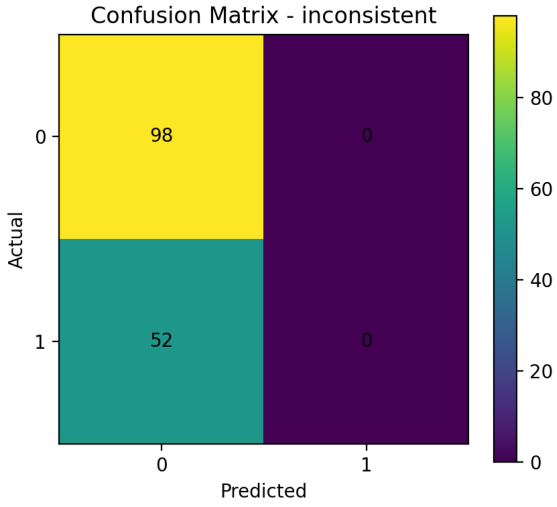
指标汇总（全量 150 样本）

标签	P	R	F1	Acc	说明
incorrect	0.909	0.365	0.521	0.387	精确率高但召回率低，漏检较多
incomplete	0.836	0.380	0.523	0.440	与 incorrect 相近，仍以漏检为主
inconsistent	0.000	0.000	0.000	0.653	未识别出正类冲突样本，需重点改进

混淆矩阵（全量 150 样本）







典型样本分析

案例 1：误判样本

样本 ID：12（Employee Card Maker）；标签：`inconsistent`；真值：1；预测：0。
误判原因：Data Safety 与隐私政策的冲突表达较隐式，模型更偏向保守预测 0。

案例 2：正确样本

样本 ID：1（Text On Photo Photo Per Status）；预测与真值均为 (incorrect=1, incomplete=1, inconsistent=0)。
关键证据：Data Safety 几乎为空，但隐私政策明确提到数据共享与数据收集。

小结

模型对“显式缺失声明”识别较稳定；对“文本冲突”类型仍需人工复核。

标签到 22 项标准的映射

- `incorrect`：优先关联第 2、9、13、20 项（缺失声明风险）
- `incomplete`：保存期限、接收方信息、用户权利路径粒度不足
- `inconsistent`：重点核查跨境传输与第三方共享冲突
- 本实验结果与 Week2 结论部分一致：声明缺失更易检出，复杂冲突仍是主要难点

关键发现

- **发现 1**：incorrect/incomplete 均为高 Precision、低 Recall，说明“命中较准但漏检明显”
- **发现 2**：inconsistent 的 P/R/F1 为 0，表明当前模型对冲突语义识别能力不足
- **发现 3**：需结合混淆矩阵解读，不能仅依据 Accuracy 判断模型可用性
- **发现 4**：现阶段流程适合做合规初筛，不适合直接替代人工审计

实验结论

- 结论 1：实验流程在 150 条样本上可复现并稳定运行，输出结构完整可评估
- 结论 2：当前表现最佳标签为 `incomplete`（F1=0.523），最难标签为 `inconsistent`（F1=0）
- 结论 3：与 Week2 的 22 项研究结论部分一致：显式缺失较易识别，复杂合规冲突更难自动检测
- 结论 4：推荐采用“LLM 初筛 + 人工复核”的工程化方案

改进方向

- 模型：继续对比 Qwen/DeepSeek，并重点优化 `inconsistent` 标签
- Prompt：加入冲突判定 few-shot 样例，明确“矛盾表达”判断规则
- 数据：扩充冲突类样本并平衡标签分布，降低模型偏向负类的风险
- 工程：增加异常重试、JSON 校验与复核队列，形成稳定审计流水线

改进结果（DeepSeek Chat，随机 20 样本）

标签	P	R	F1	Acc
incorrect	1.000	0.895	0.944	0.900
incomplete	1.000	0.571	0.727	0.700
inconsistent	0.500	0.286	0.364	0.650

- 数据来源：`results/metrics_summary_random20_deepseek_chat.csv`
- 相比全量 150 样本基线：三类标签 F1 均提升，尤其 `inconsistent` 从 0 提升到 0.364

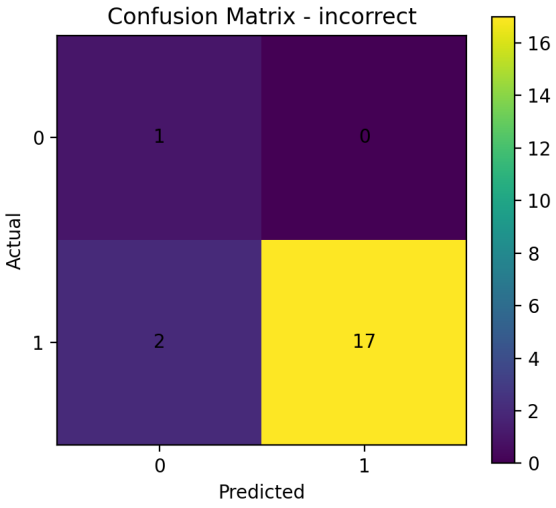
改进效果对比（F1）

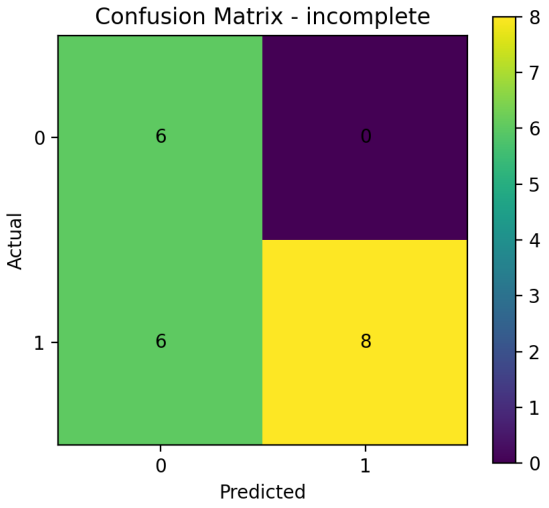
标签	全量 150 基线	随机 20 改进	提升
incorrect	0.521	0.944	+0.423
incomplete	0.523	0.727	+0.204
inconsistent	0.000	0.364	+0.364

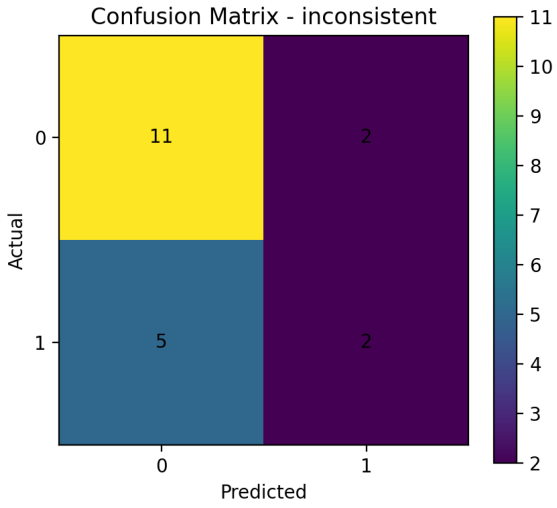
解读

few-shot 冲突样例与 `deepseek-chat` 组合后，模型对冲突标签的可识别性明显增强；但该结果基于 20 样本，仍需在更大样本上复验稳定性。

改进后混淆矩阵（随机 20 样本）







图源：results/figures_random20_deepseek_chat/

感谢聆听！